Week 5 Worked Companion

Using the worked companion

Attempt the student exercises independently first. This companion contains hints and complete solutions. Read only as far as needed, then return to your attempt.

Examples are original teaching examples. Follow each official link for the source problem and its complete contract.

Student exercises

Instructor guide

Download Week 5 code

215 · Staged hints

LeetCode 215: Kth Largest Element in an Array · Medium

H1. If the largest k seen values are retained, where is the answer?

H2. Must a new value below their minimum be retained?

H3. Heapify the first k values; replace the root only when a later value is larger.

215 · Correctness and cost

The heap retains the largest k values seen. A new value at most its minimum cannot improve that set; a larger one replaces the minimum. The final root is kth largest.

n is length. O(n log(k+1)) time, O(k) space; heapify O(k). A full negated heap takes O(n+k log n) time and O(n) space.

215 · Counterexamples and checks

In a min-heap, each parent is no greater than its children, and the root holds a global minimum. Deduplicating changes the example's answer.

`nums=[8,3,8,2,5], k=2 → 8`

Predict the state and return value before running this example. Then check the boundary cases above.

215 · Implementation

```
def kth_largest(nums, k):
    heap = nums[:k]
    heapq.heapify(heap)
    for index in range(k, len(nums)):
        value = nums[index]
        if value > heap[0]:
            heapq.heapreplace(heap, value)
    return heap[0]
```

56 · Staged hints

LeetCode 56: Merge Intervals · Medium

H1. In start order, which result interval can still overlap?

H2. May the current end shrink when a nested interval ends earlier?

H3. Sort by start; extend the last result end by max when intervals overlap.

56 · Correctness and cost

result equals the union of the processed prefix. Increasing starts mean a new interval can overlap only the last result; a gap makes a new disjoint component.

n is interval count. O(n log n) sorting plus O(n) scan; O(n) sorting workspace and output. Input order is preserved.

56 · Counterexamples and checks

Overwriting the end with 3 for [[1,8],[2,3]] loses coverage. Half-open time intervals may use a different touching rule.

```
intervals=[[5,7],[1,3],[3,6]] → [[1,7]]
```

Predict the state and return value before running this example. Then check the boundary cases above.

56 · Implementation

```
def merge_intervals(intervals):
    result = []
    for start, end in sorted(intervals, key=lambda item: item[0]):
        if result and start <= result[-1][1]:
            result[-1][1] = max(result[-1][1], end)
        else:
            result.append([start, end])
    return result
```

200 · Staged hints

LeetCode 200: Number of Islands · Medium

H1. What happens if visited land starts another component?

H2. Are diagonal neighbors connected?

H3. Start a stack traversal at each unseen land cell and mark its whole component.

200 · Correctness and cost

A traversal reaches exactly its starting component. Marked components cannot restart; the number of starts equals the component count.

R,C are dimensions. O(RC) time and space; general adjacency lists take O(V+E) time.

200 · Counterexamples and checks

Use == for values; is tests identity. Diagonal land in a 2x2 grid forms two components.

```
grid=[["1","0","1"],["1","0","0"]] → 2
```

Predict the state and return value before running this example. Then check the boundary cases above.

200 · Implementation

```
def islands(grid):
    rows, cols = len(grid), len(grid[0])
    seen, count = set(), 0
    for row in range(rows):
        for col in range(cols):
            if grid[row][col] != "1" or (row, col) in seen:
                continue
            count += 1
            stack = [(row, col)]
            seen.add((row, col))
            while stack:
                r, c = stack.pop()
                for nr, nc in ((r-1,c),(r+1,c),(r,c-1),(r,c+1)):
                    if (0 <= nr < rows and 0 <= nc < cols
                            and grid[nr][nc] == "1" and (nr,nc) not in seen):
                        seen.add((nr,nc))
                        stack.append((nr,nc))
    return count
```

994 · Staged hints

LeetCode 994: Rotting Oranges · Medium

H1. Should multiple initial sources wait for one another?

H2. When should a cell change to prevent duplicate enqueue?

H3. Enqueue all sources together, process levels, and mark a discovered cell as 2 immediately.

994 · Correctness and cost

BFS levels are minimum edge distances from any source. Equal edge durations make first discovery earliest; remaining fresh cells after exhaustion are unreachable.

R,C are dimensions. O(RC) time and O(RC) board-copy/queue space.

994 · Counterexamples and checks

[[2,0,1]] gives -1; [[0,2]] gives 0. Processing newly queued levels in the same minute undercounts time.

`grid=[[2,1,1],[0,1,0]] → 2`

Predict the state and return value before running this example. Then check the boundary cases above.

994 · Implementation

```
def rot_minutes(grid):
    rows, cols = len(grid), len(grid[0])
    board = [row[:] for row in grid]
    queue, fresh, minutes = deque(), 0, 0
    for r in range(rows):
        for c in range(cols):
            if board[r][c] == 2:
                queue.append((r,c))
            fresh += board[r][c] == 1
    while queue and fresh:
        for _ in range(len(queue)):
            r, c = queue.popleft()
            for nr, nc in ((r-1,c),(r+1,c),(r,c-1),(r,c+1)):
                if 0 <= nr < rows and 0 <= nc < cols and board[nr][nc] == 1:
                    board[nr][nc] = 2
                    fresh -= 1
                    queue.append((nr,nc))
        minutes += 1
    return minutes if fresh == 0 else -1
```

Execution and sources

```
cd code
python demo_09.py
python demo_10.py
python verify.py
```

Standard-library imports are in `solutions.py`. On LeetCode, adapt the function to the official method signature.

Erik Demaine and Srini Devadas, Introduction to Algorithms, MIT 6.006 (Fall 2011).

Checkpoint answers and reasoning

1. 8 and 2. Both copies of 8 count.
2. `[[1,8]]` and `[[1,6]]`. A nested interval cannot shorten the union; closed intervals touching at an endpoint merge.
3. `2`. Diagonals are not connected.
4. -1 and 0. The first has unreachable fresh cells; the second starts with no fresh cells.

Concept-check explanations

Insertion into a sorted list takes O(n) because elements shift, even if binary search finds the position. A min-heap is not fully sorted; its root is a global minimum.

BFS first discovery gives minimum cost when every edge has the same cost. With unequal costs, a path with fewer edges can cost more.